

华为面试笔记

华为面试Timeline

岗位：终端BG-AI技术应用

4.15 小红书上找到接口人

4.16 官网简历投递

5.12 机试&测评，华为杯数模二等奖免机试

5.22 面试 技术面&主管面

5.23 入池 符合114525定律

6.2 项目主管电话咨询（基本情况、意向度、有没有其他oc...）

6.4 项目主管说审批下来了，可以发offer

6.5 offer签约/确认后就可以选择入职报到

面试复盘：

技术面（50min）

一、自我介绍&项目介绍

（项目主要是agent&rag的一个系统、华为杯25年e题、还有一个课题项目（没问）

二、问题

Q1:你提到大模型在回答专业问题时会有幻觉，从技术原理上来说，你认为大模型为什么会产生幻觉？

A1:概率预测模型,qkv矩阵...

Q2：追问，在你的RAG系统里，你是怎么去缓解决这个幻觉问题的？

A2：要通过提供充足的上下文。检索得到最相关的K个文本片段，让大模型基于有据可查的信息去回答...

Q3:知识库和Agent规则是怎么构建和清洗的？

A3: 设定输入输出规则格式、转换为Markdown格式分级标题处理、系统提示词里加入了占位符，动态注入检索出来的背景信息...

Q4:华为杯数学建模竞赛具体介绍一下？

A4:问题的相关背景，用了什么方法，效果怎么样...

Q5:降维和提取特征后，用什么指标或方法来验证这些特征是真的有效、能拉开差距的？

A5: t-SNE降维可视化、SHAP 方法评估指标对最终故障诊断结果的贡献度排位...

Q6:既然做机器学习，你提到了样本和特征。能简单解释一下什么是过拟合吗？怎么解决这类问题？

A6:训练集上表现好，Loss低，但在测试集或实际验证时效果很差，准确率大幅下降。**解决方法：**严格划分训练集、验证集和测试集；采用交叉验证；检查特征提取；扩大样本数据集...

Q7:数据集里经常遇到正负样本极度不平衡的情况，你一般怎么处理？

A7:采用抽样技术，比如对多的一方进行欠采样，或者对少的一方进行过采样（如生成新样本），尽量让正负样本在训练时达到比例均衡.....

Q8:提到主成分分析，你能简述一下它的降维原理吗？

A8:方差贡献、碎石图保留几个主成分...没答出来具体的

Q9:追问线性代数）既然提到特征值和特征向量，你怎么理解它们的含义？作用？

A9:涉及到矩阵求矩阵向量的特征计算.....这一块具体的内容我目前有点模糊了...

Q10:了解最小二乘法吗？介绍一下

A10:最小化残差平方和，背一些公式

Q11:线性代数里面，怎么求一个矩阵的逆

A11:构造增广矩阵进行变换得到逆矩阵...（追问：还有什么方法吗？比如伴随矩阵）答伴随矩阵/绝对值，这都是考研那时候学的了，具体的有点忘记了。（面试官：没事，这些之后去查就行了）

Q12:常用编程语言？（python）说一下数据结构，比如列表和字典的区别

A12:字典是键值对，列表...（不知道怎么说了）

Q13:给你一个有重复元素的Python列表,你准备怎么做去重？

A13:用栈的思路,遍历原列表,用一个循环去判断当前元素是否已经存在于新列表中，如果不存在就append，存在就跳过

三、手撕：力扣42接雨水

没撕出来，给的案例没过。面试官一直在提示，由于时间问题结束了，简要说了下思路（应和）最后问了下时间空间复杂度。

四、反问

想问下您组里的业务主要做的是什麼？

主管面（22min）

一、自我介绍&项目介绍

（项目主要是agent&rag的一个系统、华为杯25年e题、还有一个课题项目（没问）

二、问题

Q: 介绍下华为杯的项目

A: ...

Q: 为什么用深度神经网络，这不是简单线性回归就可以了吗？用的什么神经网络

A: DANN，并介绍，主要涉及到迁移...

Q: 数据结构是怎样的？数据量多大？

A: 相当于时间序列数据.....（面试官得出结论没必要用神经网络...）频频颌首表示认同支持

Q: 你还有什么要给我看的吗？

A: 之前做了一个agent&rag的网站，也有准备ppt（没用上），您看怎么给您说一下？

Q: 一句话给我说清楚

A: 主要是垂直领域的知识库，有效解决幻觉等问题...

Q: 你主要做了什么？

A: 结合codex、claude辅助，主要是负责需求拆解、系统设计、任务边界、异常处理、是否可维护...

Q: 和豆包有什么不一样？创新点？

A: 主要是这个agent复盘能力，能结合聊天提问进行分析，给出建议...用了多agent，不同agent职责...

Q: 我懂了，这个有点太过时了，没有什么用，现在都是.....

A: 频频点头认可，

Q: 毕设准备做什么？

A: 介绍了一下，其实毕设完全没方向

Q: 平时的兴趣爱好？

A: ...

Q: 性格怎么样？

A: 乐观、积极、能抗压（照着综合测评要求去说的）

Q: 最长的实习时间？

A: 6个月？

Q: 太短了，根本干不了啥，招你干什么，最好是一年

A: 连忙改口，六个月以上，一年也行的

三、反问

您对Ai应用比较了解，想问下我们业务中是如何具体结合Ai的，您是怎么规划的
讲了下业务逻辑，成本控制（算力不够），往agent迁移

塔子哥的算法笔记（一到六）

当时一页页点有点不太方便，所以把相关的都复制下来了，具体见<https://codefun2000.com/>

一、机考

AI机考介绍（200分通过）

①考试内容：20道选择题（15道单选+5道多选，共计150分） + 两道编程题(共计450分)

②考察形式：

编程题：ACM模式（需要自行处理输入输出） + IOI模式（见下文）

选择题：多选必须选全对，少选漏选不算分。

③风格特点：出题侧重算法场景。

④选择题常见考点：线性代数，概率论与数理统计，机器学习，深度学习，强化学习，自然语言处理，大模型

⑤编程题常见考点：Kmeans,Knn,逻辑回归，DFS,BFS,排序,二分,双指针,模拟,动态规划

⑥来场真题：说再多不如自己先看一场[真题+解析](#)，看看自己能做出几道！

二、面试流程详解

机考通过后的10~20天都有可能，HR会结合你的时间和面试官的时间来安排具体面试日期

注意：请保持电话畅通，允许接听陌生电话。有些HR会通过电话而不是微信来和您确定面试时间。

面试轮次说明

日常实习共计一轮：技术一面

暑期实习共计两轮：技术一面，主管面

| **实习面试通过性：**技术面和主管面任意一面挂掉都会导致实习流程结束，无法流转继续面其他部门

秋招共计三轮：技术一面，技术二面，主管面

技术面试环节

1.自我介绍 & 工作/项目经历【0~30分钟】

面试官通常会让你做简短的自我介绍，并追问简历中的实习或项目经历。

2.基础知识问答（八股文）【0~15分钟】

八股的问答一般会基于你的项目和本岗位描述，很多时候是融汇在项目介绍里。目的不是看你有多广的八股基础储备，而是借助八股文考察你是否真的懂你的项目&是否真的具备岗位所要求的基本能力。

八股不是目的，是手段。不要盲目背，结合简历+岗位描述来。

3.手撕代码（编程题）

线上（实习与秋招中晚期的面试）

面试官会要求你**开启共享屏幕**，进行现场编码。**考察形式**通常有以下三种：

发送文字题目(80%)：

- 1.面试官通过面试会议聊天框发送题目文本或图片
- 2.同学在本地IDE编写代码，调通并通过所有样例
- 3.面试官浏览代码，询问同学代码思路。

LeetCode 题号(15%)：

- 1.让你登录自己的LeetCode账号，报题号，现场做题。

机考复查(5%)：

面试官挑一道你机考没做出来/没做满分的题目，让你当场再做一遍。

3.手撕通过凭证：正确跑出所有样例 + 思路描述正确清晰

线下

白板编程：

- 1.面试官展示题目描述

2.同学在白纸上用水性笔写代码。此时，面试官主要依靠肉眼观察你的写码过程和思维表达，**主观判断**你的编程能力和思路完整性。

4.机试复盘

一面面试官会拿到你题目的所有代码，与你交流思路、变量命名、函数作用、设计思路等。过程中可能会给你优化建议。

该环节不是100%会出现，而且要出现，只会出现在一面。

特别注意，有一种情况下你一定会有这个环节：你的代码重复率过高（第一节课的尾部详细阐述过该问题，不记得的同学可以往上翻翻）

补充说明

1.面试中考察顺序不是固定的，比如有些面试官可能一开始就让你先写代码。

2.在手撕代码环节，**每位考生通常都有一次换题机会**。如果你在当前题目上完全没有思路，可以直接向面试官说明。大多数面试官会理解这种情况，并为你**更换一道相对简单的题目**，以继续考察你的编程基础和逻辑能力。

主管面

主管面基本上就是聊生活，聊价值观，聊为人处世，当然也不排除继续聊技术项目。这个看主管的个人风格以及整个部门的氛围。

“聊天”形式（80%占比）

总体原则：回答问题的时候，不要只想着展示自己技术多强，也要让面试官感觉到你是一个好沟通、愿意配合、能吃苦、能和大家一起把事情做好的人。

1.问到缺点时

可以说一些比较常见、也比较容易改进的问题，比如有时候会有点追求完美，或者**多个任务同时来的时候，优先级安排还不够成熟**。

雷区：不要说那种会让人觉得你不好合作的话，比如不喜欢和别人交流、心情不好就想一个人待着之类的。

2.问到兴趣爱好时：

可以适当往团队协作上靠一靠，比如**喜欢和朋友一起爬山、打羽毛球、组队运动**，这些都能侧面体现你愿意和别人配合。

3.问到华为价值观时：

三点：**客户为本,艰苦奋斗,狼性文化**。不要有太多自己的想法。就算你 unwilling 加班，很不认同加班，你也别表现出来（除非你自己的能力十足的底气，有其他Offer保底。而且说实话，如果是实习，他不会让你加班的...）

以客户为中心，以奋斗者为本，长期坚持艰苦奋斗

4.反问环节：

不建议问薪资，主管一般也不负责这个。可以问问团队**主要做什么方向、平时协作氛围怎么样、目前主要在帮客户解决什么问题、团队提供的核心价值是什么**。这样会让主管觉得你不只是关注技术本身，也有团队意识和业务意识。

继续问技术（20% 占比）

一般是两种情况会继续问技术

1.部门技术氛围比较浓厚，主管本身也是偏扎实做技术的。

2.**个人经验以及项目和部门方向十分对口**，主管需要确认你是否真的能来了就干活。这种情况下，拿Offer概率还是挺高的。

不用担心的是，这个环节基本上也不会再问特别具体的技术了，就好比你的大导师问你的进展，都是从比较大的方向出发去指导。

三、面试手撕详解

Q:AI算法岗会考什么手撕？

A:总体来讲，70%可能性考察LeetCode easy~mid的原题(90%是LeetCode Hot 100)，30%可能性考察机器学习/深度学习题

影响 你所考查的手撕的题型 的因素：

1.你的简历：有些面试官会根据你**简历做过**的项目来考察（比如看你写精通机器学习，让你撕一个kmeans / 看你有Transformer相关的项目，让你撕一个注意力机制）

2.面试官所擅长的领域和偏好

PS:塔子哥和一些华为面试官交流过，他们准备面试内容的过程没有统一的标准，各自抽时间准备。所以即使是同一个小部门，你和其他人所考察的手撕都可能是不一样的，**它和面试官强相关，随机性大**

Q:手撕是ACM模式还是LeetCode核心代码模式？

A:根据我上一篇文章提到过的：1.线下手撕手写代码比较水，写代码是面试官肉眼检查，所以思路大差不差就行。2.线上手撕是**ACM模式**的，打开IDE自行写+跑样例。这里特别推荐一下我专门为大家做的：[ACM模式速通教程](#)，涵盖了求职手撕所有可能的ACM模式使用场景。

Q:ACM模式的手撕应该用什么样的代码模板？

A:

总体的原则是解耦+抽象+规范化命名。这个点许多面试官看的比较重。如果代码规范不好，很容易被面试官diss。这里塔子哥给出C++/Python/Java的推荐代码手撕模板

华为的手撕可以不需要自己写输入，可以直接将样例写死传入到函数中，如下方代码所示

PythonC++Java

代码块

```
1 class Solution:
2     def solution(self, arr):
3         """
4             核心逻辑函数
5             :param arr: 输入数组
6             :return: 计算结果
7         """
8         return sum(arr)
9
10    if __name__ == "__main__":
11        sol = Solution()
12        print(sol.solution([1, 2, 3]))
13
14    # 1print(sol.solution([1]))
15    # 0
```

Q:撕不出来会直接挂吗？

A:

这个会综合来看。如果你的bg好，项目对口，那么手撕相对会放宽一些。否则手撕一般得通过面试官所提供的所有样例+讲清楚思路才算通过。

第二个：一般面试官给的第一个题目你没思路，写不出来，一般面试官会给他准备的第二道题，是一道更简单的lc easy级别的题目给你撕。所以如果这都撕不出来，还是得反思一下适不适合吃程序员这碗饭。

此外：华为入职之后也有每年的编程考试（见第五章），所以如果真的想进入华为，建议勤加练习。

四、面试手撕高频考点Top20

26暑期实习最新手撕题统计(4月15~5.16速递)

序号	题目	频次
1	122. 买卖股票的最佳时机 II	5
2	20. 有效的括号	3
3	739. 每日温度	2
4	200. 岛屿数量	
5	583. 两个字符串的删除操作	
6	lc141 环形链表	
7	lc47 全排列	
8	Kmeans	
9	42. 接雨水	
10	160. 相交链表	1
11	56. 合并区间	
12	951. 翻转等价二叉树	
13	146. LRU 缓存	
14	1790. 仅执行一次字符串交换能否使两个字符串相等	
15	300. 最长递增子序列	
16	4. 寻找两个正序数组的中位数	
17	LCR 076. 数组中的第 K 个最大元素	
18	2289. 使数组按非递减顺序排列	
19	手撕快速排序	
20	手撕冒泡排序	
21	70. 爬楼梯	
22	Knn	
23	合并奇偶数(面试官原创题)	
24	LCR176 平衡二叉树	
25	lc322 零钱兑换	
26	lc149 直线上最多的点数	
27	lc1160 拼写单词	
28	5. 最长回文子串	
29	lc72 编辑距离	

29	1012 编辑距离
30	53. 最大子数组和
31	奇偶分区升序排列
32	叶子到根数字之和
33	Self-Attention
34	字符串相加
35	爱吃香蕉的珂珂
36	Leetcode 994.腐烂的橘子
37	Leetcode 263.丑数
38	Leetcode 179.最大数
39	Leetcode 15.三数之和
40	大模型训练MOE场景路由优化算法

24~25年高频手撕题Top20

| 统计于26，25届手撕题单

题目	考察知识点	出现次数
LeetCode 20. 有效的括号	栈、字符串	5
LeetCode 200. 岛屿数量	DFS、BFS、网格搜索	
LeetCode 1. 两数之和	哈希表	4
LeetCode 3. 无重复字符的最长子串	双指针、滑动窗口、哈希集合	3
LeetCode 46. 全排列	回溯、DFS	
LeetCode 994. 腐烂的橘子	BFS、网格搜索	
LeetCode 394. 字符串解码	栈、递归、字符串	2
LeetCode 739. 每日温度	单调栈	
LeetCode 56. 合并区间	排序、区间合并	
LeetCode 64. 最小路径和	动态规划	
LeetCode 102. 二叉树的层序遍历	二叉树、BFS	
LeetCode 1423. 可获得的最大点数 (非Hot 100)	滑动窗口、前缀和	1
LeetCode 155. 最小栈	栈、设计题	
LeetCode 986. 区间列表的交集 (非Hot 100)	双指针、区间问题	
LeetCode 456. 132 模式 (非Hot 100)	单调栈	
LeetCode 11. 盛最多水的容器	双指针、贪心	
LeetCode 49. 字母异位词分组	哈希表、字符串	
LeetCode 71. 简化路径	栈、字符串模拟	

LeetCode 217. 存在重复元素	哈希、集合
LeetCode 21. 合并两个有序链表	链表、双指针

五、华为面试手撕10天备考速通规划

刷题方法论

- 1.每道题先自己想 10~15 分钟。如果还是没思路，或者想到了大概方向但写不出来，就直接看题解。看完以后别停，自己再独立写一遍，这样才算真正学会。
- 2.**速刷最忌讳的就是死磕。**一道题卡很久，看起来很努力，其实效率不高。很多题做不出来，不是你不行，而是因为你还不知道对应的算法和技巧。有些东西确实不是硬想就能想出来的。
- 3.**前期刷题，重点是先见套路、学套路。** Hot 100 里很多题都有固定模型。刚开始学的时候，看一道不会一道很正常，不用怀疑自己。先把常见题型和解法弄明白，做多了以后，题感自然会慢慢出来。
- 4.遇到你觉得很巧、很有代表性的题，记得收藏。这种题第一次看懂，不代表以后还能自己写出来。后面要找时间多做几遍，做到下次再碰到时能自己写出来。
- 5.我们刷题的目标，是尽快把 Hot 100 这些经典题型搞懂、会写。不是为了跟题目硬耗，也不是为了证明自己多能想。该看题解看题解，把效率放在第一位
5. **“书读百遍，其意自现”**。写算法代码也是一样的道理。根据我的经验，Hot 100至少刷两遍以上，手撕阶段才能很稳定的写出来。所以过第一遍的时候，过两天忘了是很正常的事情，多刷多思考！

LeetCode Hot 100 刷题

这个部分，AI岗和非AI岗是共用的。

注意：每个人的基础不同，不一定要严格按照这个天数来。刷的快的同学可以一天刷多个Day。刷的慢的同学适当抛弃Hard题+放慢节奏。**刷题过程要注重质量，别还没理解呢就下一题了。再一个就是一定要自己写一遍代码，别只看题解代码，这样到时候很难自己独立撕出来的**

Day1：哈希,数组技巧,矩阵

[哈希](#)

[普通数组-数组技巧](#)

[矩阵四题](#)

Day2：栈与双指针

[栈-基础题](#)

[单调栈](#)

[双指针技巧-数组上的快慢指针](#)

[双指针-同向双指针](#) [双指针-相向双指针](#)

Day3：回溯

[组合型回溯](#)

[排列型回溯](#)

Day4：BFS，动态规划基础

[图论基础](#)

[BFS入门](#)

[动态规划入门](#)

Day5:动态规划进阶：背包模型与多维动态规划

[背包模型](#)

[多维动态规划](#)

Day6:贪心，二叉树基础

[Leetcode 121.买卖股票的最佳时机](#)

[Leetcode 55.跳跃游戏](#)

[Leetcode 45.跳跃游戏 II](#)

[Leetcode 763.划分子字母区间](#)

[Leetcode 104.二叉树的最大深度](#)

[Leetcode 437.路径总和III](#)

[Leetcode 543.二叉树的直径](#)

[Leetcode 124.二叉树中的最大路径和](#)

[Leetcode 102.二叉树的层序遍历](#)

[Leetcode 199.二叉树的右视图](#)

Day7:二分查找与堆

堆(Heap)

[一文彻底搞清堆\(Heap\)](#)

[堆的调库使用](#)

[LeetCode 215. 数组中的第K个最大元素](#)

[LeetCode 347. 前K个高频元素](#)

[LeetCode 295. 数据流的中位数](#)

二分查找

二分入门

通用代码模板

Hot 100 基础5题

LeetCode 4. 寻找两个正序数组的中位数 - 选做

剩下的时间

补全Hot 100中上述未涉及到的知识点章节：技巧，数学，链表（华为考这些考的相对少）

AI岗-非传统力扣刷题

除了上述考点，AI岗的同学需要额外准备以下几道题👉

手撕Kmeans

手撕Knn

手撕线性回归

手撕反向传播

SoftMax计算

Self-Attention 机制

多头注意力机制

六、面试项目准备

后面问了一些项目和实习相关内容

- 侧重挖掘难点
- 以及在做的过程中是怎么妥善解决的

1.自我介绍 & 工作/项目经历【0~30分钟】

面试官通常会让你做简短的自我介绍，并追问简历中的实习或项目经历。

2.基础知识问答（八股文）【0~15分钟】

八股的问答一般会基于你的项目和本岗位描述，很多时候是融汇在项目介绍里。目的不是看你有多广的八股基础储备，而是借助八股文考察你是否真的懂你的项目&是否真的具备岗位所要求的基本能力。

八股不是目的，是手段。不要盲目背，结合简历+岗位描述来

第一，八股不要盲目背，要**围绕项目和岗位来准备**。这场面试里，面试官主要围绕**自我介绍和项目提问**，尤其对Transformer问得很深，还要求现场画架构图并讲解。很多同学准备面试时，喜欢零散地背八股，觉得背得越多越稳。但实际面试里，问题通常不会漫无目的地发散，而是围绕你的**项目经验、实习内容和岗位要求**展开。面试官本质上是在判断：**你到底是不是真的做过这个项目，能不能进组后快速上手。**

第二，机考结束后一定要复盘，尤其是AI岗的选择题。这次面试里，面试官不仅问了编程题，还追问了那场机考里的选择题。很多人考完只看代码题却忽略了选择题，其实这部分也可能成为后续技术面的切入口。机考不是考完就结束了，复盘本身就是面试准备的一部分。

第三，手撕代码不一定是Hot100原题，但绕不开高频模型。这场手撕题是0-1背包类型，不属于Hot100原题，但依然很典型。华为线上手撕常见的形式，要么直接现场写题，要么在本地IDE写，能跑通样例，再把思路和时空复杂度讲清楚就行。所以手撕不要只看自己刷了多少题，更重要的是把高频模型真正理解透，比如动态规划、双指针、滑动窗口、二叉树递归、回溯这些。

第四，主管面不是随便聊，核心是看你**是否适合团队**。主管面更关注的，往往不是技术细节，而是你是否有**协作意识、沟通能力和稳定做事**的状态。回答缺点时，**不要把自己往“不好沟通”上引**；如果有反问环节，也可以多问**团队氛围、业务价值**这类问题。这样会让对方感觉到，你不是只盯着技术本身，也在关注**团队和业务**。

总结下来，准备华为面试时，最重要的就是四句话：**项目要讲透，机考要复盘，手撕按高频模型练，主管面要体现协作感**：

七、相关经验流程（小红书上平时刷到的经验贴都会整理下来）

1.自我介绍 & 工作/项目经历【0~30分钟】

面试官通常会让你做简短的自我介绍，并追问简历中的实习或项目经历。

2.基础知识问答（八股文）【0~15分钟】

八股的问答一般会基于你的项目和本岗位描述，很多时候是融汇在项目介绍里。目的不是看你有多广的八股基础储备，而是借助八股文考察你是否真的懂你的项目&是否真的具备岗位所要求的基本能力。

八股不是目的，是手段。不要盲目背，结合简历+岗位描述来

3.手撕代码（编程题）

技术面

40分钟

没有自我介绍与寒暄，上来直接手撕代码。本地 LeetCode，自备 IDE。无论什么题要会说正确思路

随后的项目环节。介绍的时候面试官基本不打断，之后，根据实际项目经历，针对性深挖具体技术细节。

自我介绍问几个小问题

介绍项目 问细节问题、进行阐述解释

问价值观类，性格类，学习生活类的问题

手撕一道代码题，sqrt函数，用二分法，说一下思路

屏幕共享看简历、简单自我介绍讲项目

每一个模块我都在细讲，如果redis突然崩了怎么办？

问agent项目相关的东西，文档切分方法？rag的流程，多agent协作？skills放哪个目录下，相关八股
懂不懂概率论和大模型微调的一些东西

有没有微调过模型，lora矩阵降维减少微调成本，sft和lora的区别

自我介绍-项目拷打-代码手撕

经典问题：

针对项目：1.为什么选择xx方法而不是xx 2.为什么你的实验结果可靠

项目拓展：1.落地后可以应用在哪里 2.落地需要哪些准备或具体操作

机器学习基础：1.数据太大了怎么处理 2.训练速度太慢怎么处理

平时用的啥vibecoding工具，模型是什么

GPU方面、分布式GPU是怎么计算的、TensorCore

1. 自我介绍
2. 询问项目具体做什么
3. 你平常用ai写代码多吗
4. ai写的代码你觉得怎么样
5. 你觉得现在各个领域最好用的模型分别是什么
6. 你知道SDD吗（记不清具体是哪三个字母了，完全没听过，和面试官说，跳了）
7. 你对MCP，Skill这些agent相关的领域有什么了解吗
8. 手撕代码：割绳子(二分)（完全没复习到二分，面试官人非常好，疯狂给我提示，但是拼劲全力没做出来）

业务面：

1. AI这一块用的是cursor工具吗？用的什么模型？
2. 选择千问Plus模型的原因是什么，有什么考虑？
3. 实现智能知识库问答系统时，有用AI辅助编码吗？
4. AI辅助项目开发有哪些优秀实践？遇到过AI幻觉、上下文有限等问题吗？
5. 了解过业界提升Agent编码能力的工程规范与实践吗？
6. Redis除了缓存还有哪些应用场景？
7. 多级缓存场景下，如何保证数据一致性？
8. Spring Boot有哪些特性，方便后台搭建与微服务架构开发？
9. 商用后台软件面对大流量/高并发场景，有哪些处理要点？

问题不多，主要是项目聊了很多

反问：

我问了若有幸进入该部门，实习生的优秀标准是什么

主管面

流程

40分钟

首先开场是自我介绍，需要提前准备，如果只是简单说几句，主管会看着你等着你继续说，直到你说结束语……

问：热身环节，用三个词评价自己？你的成绩怎么样？你是如何学习的？

问：项目环节，和技术面不同，主管面不太关心你用了什么技术与算法，更关心的是：你怎么干的？为什么这么干？业务场景是什么？用户视角是什么？

问：开放性灵魂拷问，如何看待未来 AI 的发展？它会取代我们吗？会代替你的同事们吗？如何看待 AI Coding？有没有风险？

问：你是否了解华为，你对华为的印象？华为的价值观？是否了解你报的部门？

1. 如何应对压力
2. 华为的企业文化

3. 你遇到的最大的挫折
4. 是否有读研计划
5. 是否了解我们的工作氛围？

面试流程：自我介绍-性格方面的提问-反问

1.这几个项目中最困难的是哪个？为什么？ 2.和同伴合作的时候有没有过意见不同的时候？ 3.对华为的核心思想有没有了解？

自我介绍起手，问了项目中哪些ai能做，哪些需要自己去做，询问古法编程代码量。

做事是否会考虑完备性（比如东南西北四个方向，考虑每个方向的情况）。

兴趣爱好是什么。

主管面：

1. 先做个自我介绍。
2. 两段项目经历都是自己规划的吗？
3. AI辅助代码整理优化具体怎么实现？
4. 你们有自己的大模型吗？当时实际使用情况如何，项目开展的思路来源是什么？
5. 给大模型增加很多约束，有没有考虑过会导致注意力下降的问题？
6. 短期记忆、对话长期记忆的设计思路从何而来，有哪些输入参考？
7. 这些AI相关知识和方案主要通过什么渠道学习？
8. 你申请的是软件开发岗位吗，专业是软件工程吗？
9. 上一段实习的关键感受和收获是什么？
10. 你下半年开始找工作吗？有读研的规划吗？
11. 大学期间最骄傲的一件事是什么？
12. 项目是独立完成还是协作完成？AI项目参与人数和分工是怎样的？
13. AI项目持续了多久？协作过程中有没有产生观点争议？
14. 项目中有没有遇到过重大争议、争吵的情况？
15. 你比较遗憾、失败或接近失败的事情是什么？
16. 克服过的比较大的困难是什么，怎么想明白的？
17. 为后续找工作做了哪些准备？
18. 有没有实操过AI辅助编码？

19. 意向工作地点是深圳，广东其他城市可以接受吗？
20. 可以实习的时间是什么时候，能实习多久？
21. 秋招意向公司有哪些，选择企业的标准和排序是什么？
22. 了解过华为的价值观和企业文化吗，有什么理解？

114525原则

反问

1. 如果想换技术方向，深造还是就业？
2. 推荐一次性实习多长时间？（表达不想短期实习的想法）
3. 如果可以实习，时间上华为会跟我们进一步协商吗？还是必须按华为给定的时间去实习？

八、面试（之前看的一些求职面试的教程视频整理的）

软能力 年轻的心态 热情 照顾更多的人

关于简历书写

一份简历就几十秒

要明白简历是给谁看

一个是 HR，一个是面试官

什么情况下看，怎么去看？

忌冗余，没有标准，要突出重点

例如：

1. **明确主题**(虽然什么都学，但每份简历都要对应**招聘的要求**)

官网岗位要求是什么就对应写什么；用简洁的话让 HR 觉得和这个岗位匹配

2. 筛选点

信息：姓名 邮箱 手机号 照片（**可以额外加上实习时长**）

背景：学校 学院 专业 时间 排版工整（再加成绩专业课那些的话就太冗余了）

技术栈：**段拆成点，分点去写**：比如熟练掌握 java，mysql **清晰明了即可**

实习经历：公司岗位时间（公司写大家都知道的）无关紧要的内容不要单独占一行

项目：**利用什么技术做了什么是实现了什么效果**

竞赛和论文：尽可能多的写，**适合培养，成长快，学习优秀**

重点看匹配程度 是否是适合培养的人

灵魂三问：

能不能体现自己和岗位匹配

能不能体现自己适合培养(自己是不是优秀的人)

能不能引导，面试官提问

三个方向一定要有**加分**

写上的东西，学习的东西理解有多深，什么程度，要**经得住问的**

问什么会什么，写**熟练掌握的**

成果 **只要从产生到应用都了解了**，那么这就是你的成果（实习期间多去观察总结别人）

实习经历，平常怎么去思考，怎么去解决问题，是不是适合培养

前两个的话做了只是做了，怎么样真正体现和别人的不同，只有**延申**

1.背景

2.阐述利用什么技术，做了什么事，取得什么成果

3.**延伸**：进一步的体现自己的思考能力，想了几种方法，为什么选择该方法，优缺点等等 就是复盘思考的能力

相关细节

一个岗位一份简历

尽量控制在一页，重点突出，不要冗余

分点写，忌连续大段

实时更新

格式统一，比如分点符号，各模块字号，各模块小标题，行间距时间对齐

和投递岗位无关的不要写投递岗位出生年月 性别老家期望城市主修课程，尽量不要两三个字占据一行，排版不好看

不要出现大量留白

各种荣誉奖项都要写

教育经历 学院不要紧跟学校后面

四六级不用写

实习

多投多面多复盘，内推继任找官网

从零0到1，先不挑内容

之后注重**业务场景，技术深度，个人提升**

八股加简历上的东西，一定要能**自圆其说**

9-11月，很多人离职，hr急着捞人

如何开启实习?如何升级实习?

第一份实习怎么开启?有了第一份下一步该怎么办?

实习层级

第一段实习

核心原则:从零0到1，先不挑内容，有实习>没实习是亘古不变的真理

第x+1份实习

业务场景，技术深度，个人提升，贪多嚼不烂

随着baseline越来越复杂，多段短时间+产出丰富的实习实际含金量可能会远小于1-2段长实习

关键

不要包装的过头，业界面试官大概率比你清楚这样的产出需要什么能力;

怎么办?

从零开始

1.简历书写

个人信息+学历学校+科研/项目经历即可，只写岗位相关的技术栈，适当包装科研/项目经历，贴近业界or能

证明自身实力。注意:简历上的东西，一定要能自圆其说

对没有实习经历/对口项目的同学来说，八股是最重要的

学生工作完全无用

2.投递/面试

优先每年9-11月暑期实习离职高峰，这段时间难度会相对低

多问师兄师姐的继任/组内直推也会让难度降低很多

经历丰富

1. 简历书写适当包装实习内容，优先写更solid、自己参与更多、产出更多的项目短实习+核心产出几乎不可能，

大多数面试官可能都会觉得你造假没必要/不熟悉的工作别写，被问到大概率扣分

2. 投递/面试多段实习要更看重组里能给自己带来的产出了，个人认为**工作内容接心>组核心**

反问很重要，组的工作≠你的工作，务必明确

反问

工作内容：具体负责哪些业务？组内的工作和个人的工作

团队构成：团队规模？组内工作基线/水位？成长空间？

项目目标：当前重点？预期产出？后续方向？

实习≥论文,重要性层级:

Match且有含金量的实习(核心模型·业务落地·量化成果)>Match的论文(研究方向对口·技术深度体现)>

不Match 的论文(方向偏离·难以转化落地)

分辨实习的含金量:核心内容VS杂活

1.高含金量

模型迭代/策略优化/参与核心搜索/推荐/广告模型搭建、重构、迭代，前沿内容探索。能接触到组内核心技术，对简历和视野提升巨大(即使实际没有收益)

2.中高含金量

业务落地/创新探索

将算法技术应用于特定业务场景，解决实际问题。有一定技术挑战，能锻炼工程能力和业务理解。

3.中低含金量

数据清洗/调参小子

辅助性、几乎没有思考的工作，缺乏核心技术挑战。可以了解业务流程，但对技术深度提升几乎没有

4.低含金量(尽量避免)

纯打杂

这个就是有点像那种让看论文流程性、重复性工作，与算法核心关联度低。对简历完全没有帮助

如何最大化实习能给自己带来的价值

1.承担"看起来很Fashion"的活

如结合大模型、创新型MoE、超长序列、因果推断等前沿技术，即使短期无直接收益，但对简历和视野提升巨大。

2.积极输出与沉淀

整理项目报告、技术总结、结构图等，形成自己的知识库。沉淀下来的内容可以成为面试素材。

周报月报、技术文档、结构图

3.主动学习与请教

充分利用公司内部资源，向同事请教业务逻辑、技术细节。实习是难得的学习窗口期。

4.简历量化

所有工作内容都用数据量化，例如”采用XX方案优化模型X，解决了xxx的弊端/业务痛点，使CTR提升Y%，带来Z收益”。数字最有说服力。

算法面试中的隐藏分：技术之外的加分项

1.表达逻辑

清晰、有条理地阐述思路，善用STAR法则(Situation,Task,Action,Result)

结构化表达STAR法则

金字塔原理

2.业务敏感度

对公司产品和业务、该方向有基本了解，能结合业务思考技术方案。

产品体验、竞品做法、业务价值

3.问题解决能力

面对开放性问题，展现分析、拆解、逐步解决问题的思路。

问题分析、方案设计、迭代优化

4.沟通与协作

团队协作意识，面对技术挑战时的积极态度。

互联网实习&&秋招核心

互联网大厂（技术驱动、快速迭代）

看重能力：

1.技术深度（算法原理·架构设计）

2.业务理解（业务逻辑·技术转化）

3.工程能力（代码质量·工程化）

4.性格人品（没有面试官不喜欢逻辑清晰、性格好的同学）

面试侧重：

算法·数据结构(手撕代码)

项目细节(技术深度挖掘)

临场解决问题能力(部分核心组会考察前沿理解哦~)

准备重点:刷题300+、项目深挖、系统设计、业务理解

核心竞争力（稳定保障·长期发展）

竞争力的要素

1.基础能力（基础决定了你的学习成本一进一步的培养成本）

2.业务理解（业务理解校招生比社招欠缺了什么?）

3.个人思考（怎么用技术解决问题为什么?）

4.综合素质（好的性格强大的沟通能力让团队协作事半功倍）

如何培养

确保实习部门、工作内容的含金量

一段长实习>多段短实习

综合素质与潜力写文档，画结构图，常复盘

期间多思考，总结出自己的东西

实习：重点是学习，而不是产出。有产出让老板开心固然重要，但能从一段实习中学到什么独属于自己的东西，才是这段实习的真正价值

总结&Q&A

核心

实习≥论文

Match且有含金量的实习是算法岗最重要的敲门砖，不match的论文只能让你进入面试流程，很难对面试/定级起到核心帮助。

价值最大化

承担Fashion的活、积极输出、主动学习、简历量化，让实习价值最大化

个人竞争力

实习就是培养个人竞争力的过程，二个人竞争力才是秋招能拿到超大包的核心因素。

寄语

希望大家都能把握住每一次机会，拿到心仪的Offer!!

记住:运气是很重要，但是托底靠的永远是实力。没有实力，机会来了也可能会接不住!

未来展望

持续学习，保持技术敏感度

深耕业务，成为领域专家

拓展视野，探索多元发展

保持热情，享受成长过程

秋招是一场马拉松，不是百米冲刺。保持节奏，持续进步，成功自然水到渠成。

九、针对自己的项目准备的问题

我面试的是华为的ai技术应用开发岗位，刚才我的面试接口人给我嘱咐了面试的一些注意事项，上午的技术面试的面试人可能不会问太多关于大模型的问题。让我好好准备简历上的项目，要本专业的、实验室的那些都要彻底搞懂讲明白。下午有技术主管面试，是华为的一个主任，主要可能会问大模型agent方面的一些内容结构框架要搞清楚，不要似是而非，能说出来

1.技术面试

知域

Q：你这个“知域”项目和普通 ChatBot 有什么区别？

答：

普通 ChatBot 更像是“用户问、模型答”，但我这个项目更强调知识库治理和 RAG 闭环。完整流程是资料上传后，先做文档检测、解析、Markdown 清洗、chunk 预览，管理员确认后再向量化入库；用户提问时，先做 query processing，再走 BM25 和 Milvus 向量检索，用 RRF 融合候选，再 rerank，最后生成带引用的回答。回答后还会记录 RAG 日志、用户反馈、badcase/goodcase 和模型成本，用于后续评估和迭代。项目文档里也明确把核心价值定义为**上传资料 -> 清洗切块 -> 向量入库 -> RAG 问答 -> 引用展示 -> 用户反馈 -> badcase/goodcase 回流 -> 评估中心**的闭环。

Q：你的 RAG 检索链路具体怎么走？

答：

我的链路是：用户问题进入后端后，先做 query rewrite 和 query expansion；然后同时进行 BM25 关键词检索和 Milvus 向量检索；两路召回结果用 RRF 融合；融合后的候选再经过 rerank 模型重排，取 top-k chunk 构造 prompt；生成阶段要求模型用固定格式输出引用标记，前端把引用标记渲染成证据链。面试项目文档里也把这条链路总结为 query processing、BM25、向量检索、RRF、rerank、引用渲染和日志评估的完整流程。

Q：为什么要 BM25 + 向量检索，不直接用向量库？

答：

因为学习资料场景里有很多专业术语、公式符号、章节名和题目关键词。BM25 对精确关键词、术语、公式片段比较敏感；向量检索更擅长语义相似，比如用户换一种说法提问。两者结合可以提高召回覆盖面。

比如用户问“查卡方表没有完全对应的 p 值怎么办”，BM25 可以匹配“卡方表、p 值、临界值”等关键词，向量检索可以召回“插值、区间判断、统计显著性”相关解释。最后 rerank 再把最适合回答的问题片段排到前面。你的项目展示里也提到，会对问题改写并向量化，基于 BM25 和语义相似查询知识块，再用 rerank 排序 top5 参考内容。

Q4：chunk 怎么切？chunk 太大太小分别有什么问题？

答：

我不是简单按固定字符数硬切，而是尽量按 Markdown 标题层级切，同时保留 title_path、页码、token_count、公式和表格标记，避免把公式、表格、题干和答案拆散。项目文档里也记录了知识库入库时会做文档检测、解析、Markdown 清洗、chunk 预览和确认入库。

chunk 太大，召回后 prompt 里会塞入很多无关内容，影响回答准确性和成本；chunk 太小，容易丢上下文，比如一道题的条件、公式和解答被拆开。我的策略是：基础按结构切分，再通过预览和 badcase 回流调整粒度。

Q5：PDF 公式识别不准怎么办？

答：

我会先区分 PDF 类型：数字原生 PDF、扫描 PDF、混合 PDF 不应该用同一种解析方式。数字 PDF 可以优先走结构化 parser，扫描 PDF 需要 OCR，混合 PDF 要结合 OCR 和文本抽取。解析后还要做 Markdown 清洗、公式和表格处理，并让管理员在入库前预览确认。项目里也明确把

“PDF/Docx/Markdown 解析、OCR/parser adapter、Markdown 清洗、chunk 预览、确认入库”作为知识库流水线的一部分。

Q6：你的 Agent 怎么设计？不是套壳吗？

答：

我把 Agent 理解为“**围绕任务拆解、工具调用、状态管理和结果校验的 engineered workflow**”，不只是让大模型多说几句话。在我的学习复盘场景里，Agent 会基于一段时间的聊天记录，分析高频薄弱点，生成知识图谱、可视化图表和复盘报告。项目材料里也有学习复盘、多 agent 和知识图谱模块，说明它不是单轮问答，而是把历史记录分析、报告生成和建议输出串起来。

更工程化地讲，我会拆成几类角色：Planner负责拆任务，Collector 负责收集聊天记录和错题数据，Analyst 负责统计薄弱知识点，Writer 负责生成报告，Critic 负责检查结论是否有依据。这样做的好处是每一步输入输出更清楚，也方便调试和替换。这些具体是怎么设计的？要不要具体说明一下？

Q7：如果让你把这个 RAG/Agent 能力落到华为终端产品里，你觉得能用在哪儿？

答：

我会优先考虑三类场景。第一类是内部研发知识助手，比如产品文档、接口文档、测试规范、故障案例的检索问答；第二类是面向消费者或售后的智能问答，比如手机、平板、PC、穿戴设备的使用问题和故障排查；第三类是研发效率工具，比如需求文档分析、测试 case 生成、日志归因和缺陷复盘。

我认为关键不是“接一个大模型”，而是要解决可靠性问题：知识库要可治理，答案要有证据链，模型调用要有 fallback，用户反馈要能沉淀为 badcase，成本和 token 要可观测。你的项目里已经有 provider registry、fallback、token usage、成本估算、RAG 指标和 badcase/goodcase 回流，这些正好可以往企业级 AI 应用可靠性方向讲。

华为杯(解决的问题、传统机器学习、DANN)

2.主管面试

Q15：你为什么想来华为？

答：

我对华为的理解是，它非常强调**技术落地、客户价值和长期投入**。终端 BG 的 AI 应用方向不是单纯做算法指标，而是要把 AI 能力集成到**真实产品、真实用户体验和真实工程链路**里。我自己的项目经历也是从**数据分析、机器学习到 RAG/Agent 工程化**逐步展开，所以我希望能在华为这种**产品场景强、工程要求高**的团队里，把自己做项目的能力转化成实际交付能力。

Q16：你最大的缺点是什么？

答：

我现在比较明显的不足是，多任务同时推进时，优先级拆分还不够成熟。之前做项目时，我有时会同时关注功能实现、页面体验、评估指标和代码细节，容易一开始摊得比较大。后来我改进的方法是先把任务拆成**必须交付、影响体验、后续优化**三层，优先保证主链路闭环，再处理细节优化。这一点我还在持续训练。

这个回答比较安全，体现能反思、能改进，不会显得不好沟通。

Q17：你怎么看 AI Coding？会不会替代程序员？

答：

我觉得 AI Coding 会明显提高开发效率，但短期不会替代能理解业务、设计架构、验证结果的人。AI 更擅长生成局部代码和提供思路，但**需求拆解、边界条件、异常处理、安全性、可维护性和最终验收**仍然需要工程师负责。比如我的知域项目里，AI 可以辅助写接口或组件，但 RAG 链路怎么设计、文档如何治理、引用怎么对齐、badcase 怎么回流，这些还是需要人做判断。

Q18：项目中和别人意见不一致怎么办？

答：

我一般会先把争议从“谁对谁错”转成“目标是什么、约束是什么、怎么验证”。比如一个方案是快速上线，一个方案是更稳但耗时更长，我会先看当前阶段目标：如果是 Demo 验证，先保证主链路跑通；如果是正式使用，就要优先考虑稳定性、数据污染和异常处理。能用实验验证的地方尽量用数据说话，不能验证的地方先做低成本试点。

反问

如果我有机会进入团队实习，AI 应用工程师在组内更偏模型应用集成、RAG/Agent 工程，还是偏数据分析和算法效果优化？我想提前对齐需要补强的能力

“团队目前在终端 AI 应用落地时，最关注的是用户体验、模型效果、端侧性能，还是工程稳定性？实习生如果想做到优秀，通常需要在哪些方面体现价值？”

十、一些agent相关的

找了比较经典的几篇论文，但没问到也没有机会说

主要是Agentic Reasoning、如何建立有效的agent系统、为什么多agent总是失败，感觉都比较经典

一、规划模块

1. 核心结论

当前 Agent 的规划能力本质上是“在人类设计好的规则轨道与代码循环中，由 LLM 负责局部决策和填空的一种半自动机制”。

2. 核心逻辑

为什么 Planning 不能全靠 LLM 完全自主完成，而是需要 Prompt + 外部代码共同实现？

因为大模型（LLM）本质上是一个**基于概率的下文预测模型**。如果没有任何外部约束，直接让它凭空生成一个复杂的长周期计划，它极易发生“幻觉”、**中途跑偏或过早终止**。

为了弥补大模型长周期规划能力的不足，目前的解决办法是“把大任务拆成一小步一小步，想一点、做一点、看一点”：

1. **人类写提示词（Prompt）**：去规范大模型的思考格式，逼它“先思考、再行动”。
2. **外部程序（如 Python 代码）**：像一个坚固的“容器”，负责拦截模型的输出，替它去调用工具、查数据库，然后把结果再喂回给模型。
3. 大模型其实是在人类搭好的**代码铁轨**上，边看环境反馈、边做下一步的局部决策。

3. Prompt 负责什么，硬编码负责什么

在工业界落地项目时，两者的边界非常清晰，类似于“软约束”与“硬轨道”的配合：

模块	负责内容（做什么）	核心优势	适用场景
Prompt 控制 (大模型的软约束)	1. 复杂的 意图拆解 与局部决策 2. 具体的 工具选择与参数推理 3. 具有创造性或需要泛化的 内容生成/话术润色	极高的 灵活性 ，能够应对模糊、非标准化的输入。	创意文案、用户模糊意图解析、根据检索结果做跨多文档的总结归纳。
硬编码控制 (传统代码的硬轨道)	1. 核心的 主干业务流程 与状态机切换 2. 权限管理 、合规风控与资金交易安全 3. 维护 Agent 的 外部主循环与工具真正执行	100% 的 确定性 与 高可控性 ，结果可预期、易审计。	身份验证、扣款交易、敏感数据过滤、Python 拦截器与主循环。

4. 主流框架对比

 主流框架规划机制一览表

框架/方法	核心思想	规划体现在哪里	LLM 负责什么	外部程序负责什么	优点与局限
ReAct	推理（Reason）与行动（Action）紧密结合，交替进行。	在线即时规划。 边想边做，根据上一步的观察动态决定下一步。	每一轮循环中，生成 Thought（思考）和决定调用的 Action（工具和参数）。	维持主循环；解析 LLM 输出的工具指令，实际调用工具并返回 Observation。	优点： 极其灵活，能根据环境反馈实时纠错。 局限： 缺乏宏观大局观，容易陷入局部死循环。
MRKL	LLM 充当“路由器”，周围挂载一组专业专家模块（如计算器、API）。	单步或多步的工具路由选择。 思考“解决这个问题当前该用哪个工具”。	理解用户意图，比对工具描述，输出标准的工具调用指令。	维护工具列表并塞进 Prompt；解析模型的路由指令并刚性执行该工具。	优点： 能完美结合传统确定性算法和外部 API。 局限： 缺乏深度的任务多步拆解能力。
BabyAGI	引入一个宏观的 任务清单（Task List）管理循环。	宏观任务列表的动态调整。 随着执行结果派生新任务并重排优先级。	充当两类 Agent： 1. 任务创建：看结果想出新子任务。 2. 任务排序：重新排先后顺序。	用 Python 代码维持整个主循环，存储并管理 Task List 数据结构。	优点： 有宏观视野，适合长周期、目标导向的复杂任务。 局限： 对外部代码依赖深；若单步出错，任务列表容易整体跑偏。
AutoGPT	类似更复杂的 BabyAGI，配有更丰富的工具链与长期记忆管理。	目标驱动的自主命令生成。 在无限循环中不断逼近设定的最终 Goal。	结合当前记忆和环境反馈，输出下一个要执行的 Command 和背后的 Reasoning。	维持主循环，刚性执行命令（如读写文件、运行代码），管理 Memory。	优点： 自动化程度极高，工具极其丰富。 局限： 极易“套娃”陷入无效死循环，Token 成本高，长序列规划易失控。

论文一、Agentic Reasoning! Reasoning LLMs with Tools for the Deep Research

1. 一句话核心结论

Agentic Reasoning 框架通过在推理大模型的超长思维链中，以特定 Token 触发并委派“网络搜索、代码运行、思维导图”三个外包子代理，实现了在非结构化、主观性强的深研任务中达到博士级分析水平。

2. 这篇文档在回答什么问题

1. **主观开放任务的瓶颈**：面对没有标准二元对错答案、高度主观的深度研究领域（如社会科学、法律金融），如何让大推理模型超越纯数理逻辑，具备严谨的综合分析能力？
2. **长思维链的工程解耦**：如何在不污染主模型上下文、不引发 Token 暴涨、不中断核心逻辑连贯性的前提下，让模型高效调度外部工具？
3. **全局逻辑的一致性**：面对动辄上万字的长周期深度推理，如何构建大模型的“动态白板”，防止其在中途遗忘关键线索或掉入逻辑陷阱？

3. 核心概念解释

大型推理模型（Large Reasoning Models）

- **通俗解释**：通过疯狂自我练习、学会了在脑子里“碎碎念一步步思考”、擅长啃硬骨头的高智商大模型（如 DeepSeek-R1、OpenAI o1）。
- **技术含义**：基于大规模强化学习（RL）训练，生成长序列内部思维链（Chain-of-Thought）以攻克复杂问题的语言模型。
- **为什么重要**：它们是多代理系统的“中央大脑”，提供了极其强大的底层逻辑底座，但其原生状态下缺乏与外部世界的深度工具交互能力。

代理化推理（Agentic Reasoning）

- **通俗解释**：高智商大脑在做深度研究时，不仅靠死记硬背，还会自主去翻书（搜索）、按计算器（代码）、在白板上理线索（思维导图）。
- **技术含义**：一种将专门处理特定任务的 LLM 代理（Agents）作为外部工具无缝集成到主推理模型生成过程中的迭代框架。
- **为什么重要**：它打破了纯模型闭卷考试的知识与算力上限，让大模型具备了在测试时通过消耗外部算力换取更高准确率的能力。

思维导图代理（Mind Map Agent）

- **通俗解释：**在后台帮你实时记笔记、画网状关系图的“记忆秘书”，防止你在长线思考时断片或被别人带偏。
- **技术含义：**基于 GraphRAG 思想，利用专门的构图 LLM 实时将主推理链抽离为结构化知识图谱，通过社区聚类算法进行主题摘要，并提供图检索支持。
- **为什么重要：**能够显式地锁死实体间的语义因果关系，显著降低模型在遭遇复杂博弈（如狼人杀）或逻辑陷阱时的犯错率。

网络搜索代理（Web-Search Agent）

- **通俗解释：**不是把百度或谷歌的几十个网页原封不动砸给大模型，而是先去查资料、然后用几句高密度大白话把核心精华提炼好的“情报员”。
- **技术含义：**在获取网络检索结果后，由专用 LLM 针对当前推理步骤进行实时内容提取和压缩，仅返回强相关的摘要文本。
- **为什么重要：**阻断了海量网页噪声污染主推理链的上下文，在保障事实准确性的同时维持了高效率。

编程代理（Coding Agent）

- **通俗解释：**主模型专属的“外包程序员”，主模型让他算个复杂公式，他默默去沙盒写完 Python 并跑出数字，最后用人话把结论汇报回来。
- **技术含义：**独立于主模型、由专用 LLM 驱动的代码生成与编译执行沙盒，强制以自然语言格式向主推理模型返回计算结果。
- **为什么重要：**让专注于宏观策略的主推理模型免于陷入调试代码语法细节的“思维中断”，大幅降低了 Token 损耗。

测试时扩展（Test-time Scaling）

- **通俗解释：**推理版的大力出奇迹——给模型更充裕的时间、让它多调用几次工具去反复核实，答案的质量就会发生质的飞跃。
- **技术含义：**在测试（推理）阶段通过增加计算开销（如密集的工具交互、集束搜索、Best-of-N 采样）来换取高难度问题准确率的现象。
- **为什么重要：**它将“工具调用频次”转变为一种可观测的隐性奖励信号，为未来通过强化学习继续微调大模型提供了全新手段。

4. 文章结构与模块关系

代码块

- 1

2

3
- [引言与动机：指出传统推理模型缺乏主观深研解释力与工具协作痛点]

|

▼

4 [核心方法论：数学建模抽象 $(o, q, e, k) \mapsto (r, a)$ 的联合概率生成机制]
5 |
6 ▼
7 [工程落地：三大代理 (Web-Search / Code / Mind Map) 的内容压缩与解耦设计]
8 |
9 ▼
10 [实验与发现：GPQA与深研任务刷榜验证 —— 提炼出 "Less is more" 等工程哲学]

- **模块关系：**文章逻辑非常严密，先抛出推理模型在非结构化领域的“原生缺陷”（模块一），紧接着建立严谨的数学映射公式（模块二），再给出优雅的分分布式小助手工程架构（模块三），最后用博士级基准（GPQA、Gemini Deep Research 对抗）的数据和“少即是多”的深刻发现（模块四）来完成闭环。

5. 最值得记住的 3 个观点

💡 观点一：“Less is More（少即是多）”的工具配置哲学

- **原文大意：**与传统 Agent 挂满几十种工具库的做法不同，研究发现只提供“网络搜索”和“编程执行”两个万金油工具，就足以覆盖绝大多数博士级复杂任务。盲目堆砌工具反而会大幅增加模型误用工具的概率，降低整体系统质量。
- **为什么重要：**它纠正了工业界“Agent 工具箱越丰富越好”的误区，指出工具的边界感和精准调用才是核心。
- **怎么复述/应用：**在实际做项目或面试时，不要吹嘘自己接了多少个 API。可以说：“我们遵循了少即是多的工程克制，把 90% 的工程精力放在了网络搜索的召回纯度和代码沙盒的确定性执行上，因为盲目增加工具往往会因为大模型的工具误用而导致系统鲁棒性断崖式下跌。”

💡 观点二：将辅助计算任务“彻底外包”给专用 LLM

- **原文大意：**主推理模型（如 DeepSeek-R1）在遇到代码或思维导图构建时，绝不在自己的推理链里直接写代码，而是抛出特殊标记，委派给 Claude-Sonnet 这种专门的子大模型去干，最后只要自然语言的执行结论。
- **为什么重要：**实现了“工种协同”，避免了主推理大模型在超长推理过程中，因为调试代码语法这种微观低级任务而导致核心“高层思维”中断或被垃圾 Token 污染。
- **怎么复述/应用：**在架构设计时，要坚决推行“大脑”和“手脚”的分离。可以用推理模型作为控制宏观大局的“智囊团”，用指令微调模型（如 Sonnet 或廉价小模型）作为负责拦截、格式化、代码生成与运行的“外包团队”。

💡 观点三：工具调用频次是天然的无监督答案筛选指标

- **原文大意：**对同一个问题进行多次平行推理，如果某条推理链中使用了更多的工具调用，往往意味着它进行了更深入的求证，答案质量更好。这种高频的工具交互可以用来做推理结果的 Best-of-N 筛选。

- **为什么重要：**它为评估开放式长文本推理提供了一个极其硬核、可以量化的客观指标，而不需要依赖玄学且高成本的“LLM-as-a-judge”。
- **怎么复述/应用：**在论文写作或面试中谈及如何筛选最优生成结果时，抛出这个高阶策略：我们可以统计 Agent 在多线程思考中调用 Sandbox 运行代码或触发 Search 校验的频次，将工具调用的丰富度作为隐式的无监督奖励信号（Reward）来对答案进行加权筛选。

论文二、如何建立有效的agent系统

1. 一句话核心结论

最成功的企业级 Agentic 系统往往抛弃了复杂的第三方框架，而是基于裸 API，通过将“提示词串联、路由、并行化”等简单可组合的代码工作流硬刚落地，坚持“只有简单方案失效时才增加系统复杂度”的克制哲学。

2. 这篇文档在回答什么问题

- **定义与边界划分：**在行业普遍概念泛化的背景下，如何严谨界定“写死代码的自动化流水线（Workflow）”与“模型动态交出控制权的自主代理（Agent）”？
- **工业落地选型：**面对直接问答、固定多步处理、动态多工具调用等不同真实业务场景，如何进行精准的技术架构选型与成本收益博弈？
- **工程架构反思：**当前流行的 Agent 重量级开发框架引入了哪些工程黑盒痛点，如何在实际生产中构建更透明、更可维护的架构？

3. 核心概念解释

工作流（Workflow）

- **通俗解释：**用硬编码绑死的流水线。
- **技术含义：**大模型与外部工具的交互完全遵循预先编排好的、确定性的代码路径。
- **为什么重要：**它在生产环境中提供 100% 的可预测性与高稳定度，适用于逻辑稳固的标准化业务，不需要给模型太多的自由发挥空间。

自主代理（Agent）

- **通俗解释：**进击的大脑，路线它自己说了算。
- **技术含义：**大模型根据当前上下文自主决策流程走向、动态挑选工具，并在整个任务执行过程中保持主动权。
- **为什么重要：**专门用来攻克步骤不可预测、非线性的开放式复杂任务（如私人助理自动跨软件比对并订机票、代码库自动重构）。

增强型 LLM (Augmented LLM)

- **通俗解释**：给大模型本体安上了“手、脚和外接大脑”。
- **技术含义**：将大模型与检索 (Retrieval)、外部工具 (Tools) 和记忆 (Memory) 无缝结合的基础构件。
- **为什么重要**：它是所有复杂工作流与 Agent 的原子构建块，现在的推理模型已经普遍具备了编写查询、选工具的内生能力。

路由模式 (Routing)

- **通俗解释**：大厂经典的“意图识别分流器”。
- **技术含义**：先利用大模型或传统分类器对输入进行分类，然后将其导向完全不同的、更具针对性的下游定制提示词或工具管线。
- **为什么重要**：能有效防止通用提示词由于妥协边缘Case而拖累整体表现，且能实现大小模型分流（简单题用 1.5B 小模型，难题用大模型）以大幅降低企业 Token 成本。

调度者-工作者模式 (Orchestrator-Workers)

- **通俗解释**：懂分工的项目经理带着一帮外包打工人。
- **技术含义**：一个中心大模型负责把复杂任务实时动态拆解，分配给多个独立的 Worker 模型实例并行执行，最后由中心模型完成结果汇总。
- **为什么重要**：与传统固定步骤的并行化不同，它具备极高的动态灵活性，适合处理在运行前完全无法预测需要多少个、何种形式子任务的场景（如一次修改多个互相关联的代码文件）。

评估者-优化者模式 (Evaluator-Optimizer)

- **通俗解释**：大模型版本的“自带严苛乙方，反复修改直到完美”。
- **技术含义**：一个模型负责生成初始回答，另一个模型依据明确标准提供严厉的改进反馈，两者在代码循环中不断迭代直到满意或触发表熔断。
- **为什么重要**：它完美拟合了人类写论文、做高阶翻译的思维模式，只要有清晰的评判标准，通过多轮迭代能让最终输出的质量发生质的飞跃。

代理-计算机接口 (ACI, Agent-Computer Interface)

- **通俗解释**：专门为了给大模型使用而设计的“键盘、鼠标和说明书”。
- **技术含义**：专门针对大模型的调用习惯进行 Prompt 调优后的工具 API 接口设计与说明文档。
- **为什么重要**：在生产环境中，ACI 的好坏直接决定了 Agent 的成败，对工具接口本身做 Prompt 工程往往比大模型做 Prompt 工程产出更高。

4. 文章结构与模块关系

- **模块一：概念廓清与选型树**（划清工作流与 Agent 边界，通过“任务确定性”、“成本延迟承受力”等指标给出选型参考）。
- **模块二：工程黑盒反思**（一针见血地指出滥用 LangChain 等现成框架带来的抽象层过多、Prompt 被隐藏、难以 Debug 的工程痛点）。
- **模块三：可组合积木库**（从基础的增强型 LLM 递进解构 5 种工业级标准工作流，并最终过渡到自主 Agent 的闭环循环）。
- **模块四：工具工程（ACI）实战**（提供优化工具接口的底层心法，强调接口设计比写提示词更关键）。
- **关系说明**：整篇文章呈**递进解耦**结构。先在战略上帮你做技术选型（模块一、二），确保你没有“为了用 Agent 而强行做 Agent”；然后在战术上给你提供标准积木（模块三）；最后在执行层教你如何把这些积木的接口磨光洗净（模块四），是一套直接针对生产环境的避坑指南。

5. 最值得记住的 3 个顶级观点

观点一：工业界落地第一原则——保持设计的简单性，能用工作流就绝不用 Agent

- **原文大意**：最成功的应用场景往往不依赖复杂框架，而是采用简洁、可组合的模式。只有当简单方案无法满足需求时，才去进一步提升系统复杂度。
- **为什么重要**：一刀切断了行业里盲目追求“多体博弈、无限自主循环”的自嗨泡沫。在企业级落地中，不可控性、Token 飞涨和高延迟是商用大忌。
- **如何复述与应用**：在面试聊到项目架构时，你可以说：“我们在架构设计上克制了盲目追求 *Autonomous Agent* 的冲动，遵循 *Anthropic* 提倡的简单可组合原则。我们发现 90% 的业务都能通过‘路由+提示词串联’这种确定性极高、零幻觉的工作流锁死，这在生产环境中极大地保障了上线质量与成本可控。”

观点二：现成 Agent 框架引入了多余的抽象层，是 Debug 阶段的灾难

- **原文大意**：LangChain/LangGraph/Dify 等框架虽然降低了新手上手门槛，但它们引入了额外的抽象层，导致底层的真实 Prompt 和 Response 被隐藏，大幅增加了排错难度。
- **为什么重要**：道破了无数开发者在生产环境中使用复杂开源框架后，遇到模型抽风却“根本无法看清发生了什么、无从下手改 Prompt”的工程痛点。
- **如何复述与应用**：当面试官问你“你项目里用了什么 Agent 开源编排框架”时，最硬核的回答是：“我们在初期尝试过开源框架，但为了追求工业级的透明度与彻底掌控底层 Prompt，我们在生产环境中选择减少抽象层，直接基于裸 API 和几行 Python 代码来自研轻量级的状态机和工作流组合。这让我们能对每一次 *Tool use block* 进行刚性拦截和精确调试。”

观点三：在构建 Agent 时，优化工具接口（ACI）比优化模型 Prompt 更有用

- **原文大意：**Anthropic 团队在为 SWE-bench 刷榜构建代码 Agent 时，花在优化工具设计上的时间比优化提示词还多。如果工具对人类开发者来说晦涩难懂，模型也同样会觉得难用，甚至频繁格式报错（例如算错 diff 行数、JSON 转义失败）。
- **为什么重要：**揭示了 Agent 调优的核心秘密——不要寄希望于用一两句神奇的“小作文提示词”去纠正模型的行为，而应该通过简化接口格式、消除格式负担（如强制改相对路径为绝对路径）来为模型减负。
- **如何复述与应用：**在写简历或面试彰显深度调优功底时，抛出这个实战案例：“我们项目最大的性能跃升不是靠改 Prompt，而是我们重构了 API 接口。当时模型在生成多行 JSON 代码时经常发生转义报错，我们果断放弃了让它输出复杂的结构化 JSON，而是重构了底层工具，改由模型输出带 Markdown 标记的裸文本，由我们的后端代码去动态拦截并做字符串解析。这彻底消除了模型的格式负担，让工具调用成功率暴涨。”

论文三、为什么多Agent系统总是失败

6. 一句话核心结论

多智能体系统（MAS）当前居高不下的生产环境失败率（动辄超过 60%），不能简单甩锅给底层大模型（LLM）能力不足，其本质是由**系统架构决策失误（规范缺陷）、智能体间协同错配以及流于表面的任务验证**共同导致的系统设计顽疾。

2. 这篇文档在回答什么问题

- **失效的底层机理：**面对多智能体系统在真实高难度任务中频繁卡死或交付垃圾产物的现状，是否存在一套统一、无偏的系统级失效分类框架来精准定义它们的“死因”？
- **质量控制的盲区：**为什么即使在 Agent 团队中配置了显式的“验证器（Verifier）”角色，整个系统依然会输出带有严重 Bug 的不可用结果？
- **开发范式的革新：**当系统失败时，开发者如何从“临时的、头痛医头”的 Ad-hoc 盲目调试，转向数据驱动的量化诊断与定向工程干预？

3. 核心概念解释

MAST 故障分类法 (Multi-Agent System Failure Taxonomy)

- **通俗解释：**多智能体系统的“错题本与标准死因诊断书”。
- **技术含义：**首个利用扎根理论（Grounded Theory）自下而上归纳建立的结构化失效分类学，涵盖 3 大 Overarching 类别与 14 种细粒度失败模式。
- **为什么重要：**它统一了行业对多 Agent 失败的定性描述，划清了各模块间级联错误的起源界限，让系统化诊断成为可能。

规范缺陷 (Specification Issues / FC1)

- **通俗解释**：起跑线上的系统基因缺陷。
- **技术含义**：指由于系统架构决策失误、角色 Prompt 提示词含糊或长线状态/循环管理失效，导致 Agent 无法遵循行为规范的类别。
- **为什么重要**：在所有失败案例中占比最高（41.77%）。其中“步骤重复（17.14%）”是最大痛点，通常由于死板的轮次配置导致 Agent 陷入原地转圈的死循环。

💡 智能体间协作失调 (Inter-Agent Misalignment / FC2)

- **通俗解释**：智能体团队在动态执行中的“沟通断层”与信息孤岛。
- **技术含义**：指多智能体在交互过程中因推理与行动不匹配、不求澄清或信息传递不对称而导致的协同失败。
- **为什么重要**：最典型的是“信息扣留 (Information Withholding)”。例如，下游工具 Agent 明明通过 API 报错发现了正确用户名应该是电话号码而非邮箱，但它回传给上游主管 Agent 时，却只敷衍地汇报了一句“凭据无效，请重新输入”，彻底扣留了关键线索，导致团队无限重试直至崩溃。

🛡️ 任务验证问题 (Task Verification / FC3)

- **通俗解释**：流于形式、走过场的“面子工程”质检。
- **技术含义**：指在任务后置质量控制阶段，由于验证不全、验证错误或过早终止，导致无法捕获并纠正错误输出的失效。
- **为什么重要**：论文揭示了当前绝大多数 Agent 验证器只做表面检查（如代码能否编译、注释全不全），根本无法评估是否达成高层深层业务目标。

📊 柯恩卡帕系数 (Cohen's Kappa / κ)

- **通俗解释**：剔除“碰巧和瞎猜”之后的真实铁证一致度。
- **技术含义**：用于衡量两位独立标注者对离散分类结果达成一致性的统计量，公式为 $\kappa = \frac{P_o - P_e}{1 - P_e}$ （ P_o 为实际一致比例， P_e 为偶然一致比例）。
- **为什么重要**：论文中利用该指标评估 o1 推理模型充当裁判 (LLM-as-a-judge) 的可靠性，Few-shot 设定下 $\kappa = 0.77$ ，证明自动化失效标注已高度逼近人类专家水平。

⌚ 效率低下 (Inefficiencies)

- **通俗解释**：能一车拉完的数据，Agent 非要开着空车来回跑十趟的“磨洋工”现象。
- **技术含义**：系统虽然最终完成了正确性指标，但在执行过程中调用了冗余的多轮交互，产生了高昂的 Token 成本和长延迟。
- **为什么重要**：例如获取播放列表前 10 首歌，Agent 竟然跟 API 交互了 10 轮每次只拿一首，哪怕 API 支持单次批量获取。这是多智能体系统商用落地前必须要跨越的下一代性能鸿沟。

4. 文章结构与模块关系

- **模块一：摘要与灾难现状诊断：**揭示主流 MAS 在复杂软件、交互、数学等基准上，失败率普遍动辄超过 60% 的惨烈现状。
- **模块二：MAST 方法论构建：**利用扎根理论从 HyperAgent、ChatDev 等 5 个主流系统的运行轨迹中抽丝剥茧归纳分类，通过三轮 IAA 一致性实验洗净定义，并开发 o1 自动标注器。
- **模块三：三大失效类别解构：**全景解剖规范缺陷（FC1）、协同失调（FC2，引入经典的登录信息扣留流向图）与任务验证（FC3）的根本表现。
- **模块四：工程干预与系统设计首要性：**通过微调拓扑架构和引入高层多级验证，使系统在不升级大模型底座的情况下，成功率大幅跃升。

模块间的关系：整体呈现“发现灾难现状（模块一）->打造无偏诊断工具（模块二）->分层解剖死因（模块三）->给出系统级治疗方案（模块四）”的严密切闭环。

5. 最值得记住的 3 个顶级观点

💡 观点一：“未遵循规范”是架构设计师的失职，绝非纯模型的智商缺陷

- **原文大意：**智能体系统追求的是“自主性”（Agency）。一个优秀的 MAS 应该具备根据模糊的高层规范推断隐含细节的能力，而不是依赖人类写出几千字、毫无容错的死板 Prompt。论文通过仅仅修改 ChatDev 的角色拓扑和组织规范（在基础模型和用户指令完全不变的前提下），让成功率直接绝对提升了 9.4%。
- **为什么重要：**打破了“模型不听话就一味堆砌 Prompt 屎山”的低级开发思维，证明了系统架构重构可以完全独立于 LLM 升级来压榨出系统性能。
- **面试/项目如何复述：**“当我们的 Agent 团队在复杂场景中发生意图偏移时，我们克制了去反复修饰 Prompt 的盲目行为。我们参考了先进的组织系统论，将复合型的重度职责 Agent 解耦拆分为单职能的微服务智能体节点，从‘系统架构和通信拓扑’层面彻底解决了解释规范失效的结构弱点。”

💡 观点二：流于形式的低级检查，是验证器无法成为万能药的祸根

- **原文大意：**配有显式检查角色的系统虽然整体错误率较低，但在经典复杂任务上（如实现井字棋、数独）的正确率常常只有惊人的 33%。因为当前的验证器往往在做编译、格式等“表面检查”，缺乏更深层次的目标达成度验证。通过引入一个专注于“高层总体任务目标”的多级模块化单元测试，任务成功率瞬间绝对提升了 15.6%。
- **为什么重要：**指明了多智能体质检模块的阿喀琉斯之踵——必须像复杂软件工程一样，建立多层次、模块化的质检防线。
- **面试/项目如何复述：**“我们在搭建质检模块时，拒绝让大模型做笼统的‘好坏问答’。我们构建了多级验证流水线（Multi-Level Verification）：先用刚性代码硬编码进行输出 JSON 规范和边界断言的低级别检查，最后才让高级业务 Agent 结合行业外部知识库去验证高层目标的完整度，用模块化测试为 Agent 兜底。”

💡 观点三：隐蔽的信息扣留（Information Withholding）是引发长序列死循环的“隐形杀手”

- **原文大意：**下游执行代理在遇到错误时，往往只向上游汇报一个空泛、冰冷的错误状态码或笼统的失败结论，却将底层捕获到的具体字段约束、报错明细隐藏起来。这直接导致上游指挥官由于缺乏核心反馈，只能盲目按原策略不断重试，产生巨大的级联级灾难。
- **为什么重要：**它从“通信协议（Communication Protocol）”的角度，解释了为什么多 Agent 交互会频繁陷入无限套娃和超时锁死。
- **面试/项目如何复述：**“为了杜绝智能体多步协同中的无限无效重试，我们自研了一套‘强上下文透传通信协议’。在通信管道中，我们严禁子代理私自吞掉底层的 Traceback。任何一个子节点失败，必须将异常元数据（Metadata）结构化打包、无损透传给决策主代理，确保信息不被扣留，从而引导团队快速自我修正。”

6. 30 秒口头复述版本

“这篇关于‘多 Agent 为什么总是失败’的论文彻底颠覆了行业认知。它用实证数据证明：**多智能体落地的失败率之所以高达 60% 以上，锅在系统设计，不在大模型智商。**

论文提出的 **MAST 分类法** 抓住了三大核心死因：第一是**规范问题**，轮次配置死板导致不停地原地步骤重复；第二是**协同错配**，特别是子代理私自把具体的报错线索给‘扣留’了，只跟主管汇报一句‘失败了’，导致团队盲目重试陷入死循环；第三是**验证走过场**，只验代码能不能跑，不验业务逻辑对不对。所以，要做出真正能商用的多 Agent 系统，核心根本不是去死磕模型或写小作文 Prompt，而是去参考复杂软件系统，搞**‘架构角色解耦、强错误透传协议、以及多级模块化目标验证’**。”